1주차: 해시와 두 포인터

센서 로그의 가까운 중복

같은 이벤트가 재전송되어 [4, 7, 4, 4]가 기록되었습니다. 같은 값이 한 칸 이내에 다시 나타나는지 확인하려고 합니다. 첫 번째 4와 세 번째 4만 비교하면 마지막 두 4를 놓칩니다.

매번 모든 과거 기록을 다시 읽으면 로그가 길어질수록 비교가 빠르게 늘어납니다. 이번 차시에서는 질문에 필요한 과거 정보만 저장하고, 정렬된 입력에서는 답이 될 수 없는 후보를 제거합니다.

1주차의 문제와 학습 목표

1차시는 배열·집합·사전의 연산 비용을 비교하고, 값과 이전 위치를 연결합니다. 2차시는 정렬된 값에서 두 포인터의 이동을 정당화하고 세 값의 합으로 확장합니다.

- 입력·출력·제약을 먼저 정의합니다.
- 저장한 상태의 의미와 후보를 버리는 근거를 설명합니다.
- Python 구현의 시간·보조 공간·출력 비용을 구분합니다.

실습은 LeetCode 1, 219, 167, 15입니다.

리스트·집합·사전의 정보

리스트는 위치별 값과 중복을 보존합니다. 집합은 값의 존재 여부를 저장하며, 사전은 키 하나에 값 하나를 연결합니다. 같은 로그라도 어떤 질문에 답하는지에 따라 저장 구조가 달라집니다.

```
events = [4, 7, 4, 4]
seen = {4, 7}
latest = {4: 3, 7: 1}

print(events[2])        # 4
print(4 in seen)        # True
print(latest[4])        # 3
```

입력 크기와 연산 비용

n은 기록 수입니다. 다음 비용은 고정 폭 정수와 Python의 배열·해시 컨테이너를 기준으로 합니다.

작업	비용	이유
리스트 인덱스	O(1)	위치를 바로 읽음
리스트에서 값 찾기	O(n)	앞에서부터 비교
dict/set 조회·갱신	평균 O(1)	키로 위치를 찾음; 최악 O(n)
q개 슬라이스·복사	O(q) 시간·공간	새 참조 목록 생성
n개 정렬	최악 O(n log n)	Python 정렬 작업 공간 O(n)

Big-O는 입력이 커질 때의 worst-case 를 기준으로 계산된 상한입니다.

해시 키와 객체 복사

정수·문자열과 해시 가능한 원소만 포함한 tuple은 키로 사용할 수 있습니다. list는 변경 가능하므로 키가 될 수 없습니다. `dict.values()`는 view이고 `list(...)`가 목록을 만듭니다.

```
latest = {4: 2}
snapshot = latest.copy()
latest[4] = 3
print(snapshot[4], latest[4])  # 2 3

key = (2, 0, 1)
counts = {key: 5}
print(counts[key])            # 5
```

키가 긴 문자열·tuple이면 키 계산 비용도 별도로 셉니다. 이 주의 정수 키는 그 비용을 상수로 봅니다.

LeetCode 1: 문제 정의와 단순 풀이

Two Sum: 서로 다른 두 위치의 합이 target인 인덱스를 반환합니다. 원문은 답 하나를 보장하며 같은 위치를 두 번 사용할 수 없습니다.

`nums=[4,8,3], target=7`의 답은 `[0,2]`입니다. 모든 쌍을 검사하면 추가 저장 없이 풀 수 있습니다.

```
def two_sum_brute(nums, target):
    for left in range(len(nums)):
        for right in range(left + 1, len(nums)):
            if nums[left] + nums[right] == target:
                return [left, right]
    return []
```

쌍의 개수와 반복 조회

서로 다른 두 위치의 쌍은 다음만큼 있습니다. n=10,000이면 49,995,000쌍입니다. 실제로는 답을 일찍 찾을 수도 있지만 최악에는 이 수에 비례해 비교합니다.

$$\frac{n(n-1)}{2}$$

현재 값이 3이고 target이 7이라면 필요한 과거 값은 4 하나입니다. 모든 과거 값을 다시 비교하는 대신, 4의 존재 여부와 위치를 바로 조회할 상태를 만듭니다.

Two Sum의 이전 값 상태

`seen[value]`는 이미 지난 위치 하나입니다. 현재 위치를 읽기 전에 저장하지 않으므로 사전에서 찾은 위치와 현재 위치가 다릅니다.

```
def two_sum(nums, target):
    seen = {}
    for index, value in enumerate(nums):
        if target - value in seen:
            return [seen[target - value], index]
        seen[value] = index
    return []  # Defensive extension: official inputs have one answer.
```

Two Sum의 상태 추적

index	value	필요한 값	조회 전 seen	결과
0	4	3	{}	없음 → 4:0 저장
1	8	-1	{4:0}	없음 → 8:1 저장
2	3	4	{4:0,8:1}	발견 → [0,2]

저장된 값의 개수는 지금까지 읽은 서로 다른 값의 수입니다. 사전에는 미래 위치가 들어 있지 않습니다.

Two Sum의 정당성과 갱신 순서

반복 시작 때 seen은 이전 위치만 포함합니다. 답의 두 번째 위치에 도착하면 첫 위치의 값이 저장되어 있으므로 답을 찾습니다.

`nums=[3,3], target=6`에서 현재 값을 먼저 저장하면 조회 결과가 현재 인덱스 자신이 됩니다. 첫 3에서는 조회만 실패하고 저장한 뒤, 둘째 3에서 `[0,1]`을 반환해야 합니다.

n개를 한 번씩 읽으므로 평균 시간 O(n), 보조 공간 O(n)입니다. 일반 해시 충돌의 최악 상한은 $O(n^2)$입니다.

LeetCode 219: 거리 조건의 추가

Contains Duplicate II: 같은 값의 서로 다른 두 위치가 거리 k 이내인지 판정합니다.

`nums=[4,7,4,4], k=1`은 마지막 두 위치 때문에 True입니다. k=0이면 서로 다른 위치를 만족할 수 없습니다.

각 위치에서 앞의 최대 k개를 다시 보면 O(n·min(n,k)) 시간입니다. 슬라이스를 만들면 반복마다 복사 비용도 듭니다. 값마다 어떤 과거 위치 하나를 남기면 충분한지 살펴봅니다.

가장 최근 인덱스의 의미

`latest[value]`는 그 값이 마지막으로 나타난 위치입니다. 현재 위치와 가장 가까운 과거 위치이므로 이것도 너무 멀면 더 오래된 위치는 모두 실패합니다.

index	value	이전 latest[value]	거리	처리
0	4	없음	-	4:0 저장
1	7	없음	-	7:1 저장
2	4	0	2 > 1	실패 후 4:2로 갱신
3	4	2	1 <= 1	True 반환

최근 인덱스의 구현

```
def nearby_duplicate(nums, k):
    latest = {}
    for index, value in enumerate(nums):
        if value in latest and index - latest[value] <= k:
            return True
        latest[value] = index
    return False
```

조건을 만족하지 않은 재등장도 최근 위치로 갱신합니다. 최초 위치만 남기면 예시의 마지막 쌍을 찾지 못합니다.

최근 상태의 정당성과 저장 비용

현재 위치 j에서 같은 값의 이전 위치 i 중 가장 큰 i가 거리 j-i를 최소화합니다. 최근 위치가 k 이내면 답이며, 아니면 이전 위치 어느 것도 답이 아닙니다. 갱신 후 다음 반복에서도 같은 의미가 유지됩니다.

n은 길이, d는 서로 다른 값의 수입니다. 평균 시간 O(n), 보조 공간 O(d)이며 d<=n입니다. 이는 오래된 키를 지우지 않는 현재 구현의 비용입니다.

확인 질문: 값 4가 여러 번 나오더라도 인덱스를 모두 저장할 필요가 없는 이유를 설명합니다.

1차시 이해 확인

- `4 in events`와 `4 in latest`는 각각 무엇을 검사하나요?
- `[3,3]`에서 조회와 저장 순서가 바뀌면 어느 시점에 잘못되나요?
- `[4,7,4,4], k=1`에서 2번 위치의 갱신을 생략하면 어떤 답을 놓치나요?
- 매 반복에서 `latest.copy()`를 만들면 복잡도는 어떻게 달라지나요?

답을 먼저 적고 교사용 확인 답과 대조합니다.

정렬된 부품 길이의 합

정렬된 길이 `[1,4,6,9]`에서 합이 7인 두 부품을 찾습니다. 왼쪽 값 1과 오른쪽 값 9의 합은 10입니다. 지금 9를 버려도 되는 이유를 설명할 수 있나요?

2차시에서는 정렬 순서가 한 번에 여러 후보를 지우는 근거임을 보입니다. 포인터 이동을 암기하기 전에 제거되는 모든 쌍을 확인합니다.

LeetCode 167의 입력 계약

Two Sum II - Input Array Is Sorted: 입력은 비내림차순이며 답 하나가 보장됩니다. 서로 다른 두 원소를 사용하고 결과 위치는 1부터 셉니다. 추가 공간은 상수여야 합니다.

```
numbers=[1,4,6,9], target=7 → [1,3]
```

이 문제는 보조 공간 O(1)을 요구합니다. 이전 값을 사전에 저장하는 방식은 최대 O(n) 공간을 사용하므로, 정렬된 입력에 두 포인터를 적용합니다.

합이 작은 왼쪽 후보의 제거

현재 후보 범위는 `[left,right]`입니다. 양 끝 합이 target보다 작다면, 현재 left와 짝지을 수 있는 모든 값은 right의 값 이하입니다. 따라서 그 합들도 모두 너무 작습니다.

left를 하나 늘리면 현재 left를 사용하는 실패 쌍만 제거합니다. right를 줄여서는 부족한 합을 키울 수 없습니다.

불변식: 아직 찾지 못한 답이 있다면 현재 범위 안에 두 위치가 모두 남아 있습니다.

합이 큰 오른쪽 후보의 제거

양 끝 합이 target보다 크다면, 현재 right와 짝지을 수 있는 모든 값은 left의 값 이상입니다. 따라서 그 합들도 모두 너무 큽니다.

현재 쌍	합	target=7의 판정	제거되는 후보
1,9	10	너무 큼	9와 1·4·6의 쌍
1,6	7	정답	더 제거하지 않음

같은 원소를 두 번 쓸 수 없으므로 반복 조건은 left<right입니다.

두 포인터의 구현

```
def two_sum_sorted(numbers, target):
    left, right = 0, len(numbers) - 1
    while left < right:
        total = numbers[left] + numbers[right]
        if total == target:
            return [left + 1, right + 1]
        if total < target:
            left += 1
        else:
            right -= 1
    return []
```

루프 안에서는 0-based 인덱스를 쓰고 반환할 때만 1을 더합니다.

포인터의 종료와 전체 이동 수

매 반복에서 답을 반환하거나 포인터 하나가 안쪽으로 한 칸 움직입니다. 두 포인터의 간격이 줄어들므로 무한 반복하지 않습니다.

left는 오른쪽으로만, right는 왼쪽으로만 움직입니다. 이동 횟수의 합이 n에 비례하므로 시간 O(n), 보조 공간 O(1)입니다.

반례: 정렬 전제 밖 `[3,1,4], target=5`에 같은 규칙을 적용하면 1과 4의 답을 놓칩니다. 이 규칙은 원문이 보장한 정렬 조건에 의존합니다.

LeetCode 15: 세 값과 중복 조합

3Sum: 서로 다른 세 위치의 합이 0인 값 조합을 중복 없이 반환합니다. 값이 같아도 위치가 다르면 사용할 수 있습니다.

`[-2,0,1,1,2] → [[-2,0,2],[-2,1,1]]`

세 위치를 모두 열거하면 $O(n^3)$입니다. 값을 하나 고정하면 나머지 두 값의 목표 합은 고정 값의 음수가 됩니다.

정렬과 고정 값의 역할

입력을 복사 정렬하고 index를 첫 원소의 위치로 고정합니다. left=index+1, right=n-1로 두면 세 위치가 항상 다릅니다.

현재 세 값의 합이 음수면 left를 늘리고, 양수면 right를 줄입니다. 167과 같은 정렬 기반 제거입니다.

3Sum은 값 조합을 반환하므로 정렬한 배열에서 탐색합니다. 원래 인덱스를 반환하는 문제에서는 값과 원래 인덱스를 함께 저장해야 합니다.

3Sum에서 고정 값 -2의 추적

index,left,right	세 값	합	동작
0,1,4	-2,0,2	0	[-2,0,2] 저장; 양쪽 이동
0,2,3	-2,1,1	0	[-2,1,1] 저장; 양쪽 이동
0,3,2	범위 끝	-	고정 값 -2의 탐색 종료

그다음 고정 값은 0입니다. 값 1·1·2에서 두 값을 골라도 0의 음수인 0을 만들지 못합니다. 현재 고정 값에서 찾지 못한 조합과 다음 고정 값의 조합을 구분합니다.

3Sum 구현: 고정과 후보 이동

아래 구현과 다음 페이지의 코드는 이어지는 하나의 함수입니다.

```
def three_sum(nums):
    values = sorted(nums)
    result = []
    for index in range(len(values) - 2):
        if index and values[index] == values[index - 1]:
            continue
        left, right = index + 1, len(values) - 1
        while left < right:
            total = values[index] + values[left] + values[right]
            if total < 0:
                left += 1
```

3Sum 구현: 저장과 중복 건너뛰기

```
            elif total > 0:
                right -= 1
            else:
                result.append([values[index], values[left], values[right]])
                left += 1
                right -= 1
                while left < right and values[left] == values[left - 1]:
                    left += 1
                while left < right and values[right] == values[right + 1]:
                    right -= 1
    return result
```

정답을 저장한 뒤 양쪽 포인터를 이동하고, 방금 사용한 값과 같은 값을 건너뜁니다.

중복 제거의 두 위치

첫째, 이미 고정한 값과 같은 값을 다시 고정하지 않습니다. 둘째, 답을 찾은 뒤 같은 left 값과 right 값이 반복되는 것을 건너뜁니다. 같은 값도 서로 다른 위치에 있으면 함께 사용할 수 있습니다. 중복 제거의 대상은 이미 반환한 값 조합입니다.

`[0,0,0,0]`에서는 위치 조합이 여러 개여도 값 조합 `[0,0,0]` 하나만 반환합니다. `[-2,0,1,1,2]`의 `[-2,1,1]`은 서로 다른 두 1의 위치를 사용하므로 유효합니다.

3Sum의 정당성과 비용

정렬된 조합마다 가장 작은 값이 하나 있습니다. 그 값을 고정한 차례의 두 포인터 탐색이 나머지 쌍을 포함하고, 합의 방향으로 실패 후보만 제거합니다. 중복 건너뛰기는 이미 기록한 같은 값 조합만 없앱니다.

n은 입력 길이, z는 출력 조합 수입니다. 복사 정렬은 $O(n \log n)$, 고정 값마다 포인터 이동 합은 $O(n)$이므로 전체 시간은 $O(n^2)$입니다. 정렬·입력 복사의 보조 공간은 $O(n)$, 결과 저장은 $O(z)$입니다.

문제 조건에 따른 상태 선택

질문	남길 상태	확인할 조건
정렬 안 된 두 값의 합	이전 값→위치	같은 위치 재사용 금지
가까운 같은 값	값→최근 위치	거리 k; 실패 후에도 갱신
정렬된 두 값의 합	left,right	정렬·상수 공간·1-based 반환
중복 없는 세 값의 합	고정 위치와 두 포인터	서로 다른 위치·값 조합 중복 제거

자료구조 이름을 먼저 고르기보다, 다음 판단에 필요한 정보와 버려도 되는 후보를 먼저 적습니다.

독립 실습과 제출물

학생 실습에서 LeetCode 1, 219, 167, 15를 풉니다. `student_week_01.py`에 코드를 작성하고 작은 입력을 먼저 손으로 추적합니다.

각 문제에 상태의 의미, 정당성, 시간·공간 분석과 경계 입력을 함께 제출합니다. 막힐 때는 단계별 힌트·해설을 필요한 단계까지만 확인합니다.

Python 실습 묶음에서 `python week01_walkthrough.py`로 강의의 예시와 추적 결과를 확인합니다.

핵심 참고자료

Erik Demaine·Srini Devadas, Introduction to Algorithms, MIT 6.006 (2011년 가을 학기).

문제의 전체 입력 조건은 각 LeetCode 공식 문제 링크에서 확인합니다. 다음 주에는 연속 구간의 상태를 갱신하는 슬라이딩 윈도우와 가장 최근 미해결 정보를 보관하는 스택으로 이어집니다.